RoboBIM: Scaling Robot Semantic Foundations through Agentic BIM Priors

Abstract— **Building Information Models (BIM), typically stored in IFC format, already encode rich spatial and semantic priors for indoor navigation. Most IFC-to-robot pipelines use static one-pass conversions that produce occupancy maps or simulation assets while leaving much of the semantic information unused. We present RoboBIM, an IFC-to-navigation-graph pipeline that combines a deterministic geometric backbone with a tool-augmented large language model (LLM) agent. The backbone extracts a three-layer graph consisting of per-storey occupancy grids, a waypoint graph, and a room-transition topology. The agent then analyzes the IFC through controlled code execution to tune extraction parameters, infer room types, assign hazard flags, and estimate traversability. We also propose an online query agent that uses the same tool interface during robot operation to retrieve detailed prior information from the IFC model and enriched graph as the robot moves. We evaluate on four public IFC models. Compared with the baseline, RoboBIM increases room classification coverage, adds hazard annotations in rooms conmpared to baseline which has none, and produces traversability scores for all navigable rooms, while preserving connected topology in all cases. The enriched graph is exported in ROS2 navigation formats and also as a natural-language summary for informing on-board reasoning agents for scalable robot autonomy.**

I. Introduction

Autonomous robots in structured environments are typically asked to build their own metric-semantic representation from onboard sensing [1], [2], even when a high-fidelity digital twin of the same building already exists as an IFC schema [3]–[5]. This discards architectural priors that the robot would otherwise spend minutes re-discovering. But the converse extreme; using IFC as it is – produces priors that are geometrically complete yet semantically thin: a room is a closed volume, a door is a rectangle, and the fields a downstream planner actually wants ("is this floor slippery?", "is this corridor high-traffic?", "can my wheelbase clear this door?") are either missing or hidden in heterogeneous IfcPropertySets that cannot be parsed with plain code.

The IFC semantic gap. The deterministic IFC-to-graph pipeline – IfcSpace extraction, generalized Voronoi diagram (GVD) segmentation, door-waypoint crossing, ROS 2 serialization – is well-understood [6]–[10]. It reliably yields occupancy grids, a topological room-transition graph, and a waypoint map. What it cannot yield is the interpretive layer above that: room function when `IfcSpace.LongName` is blank, floor material when materials are attached to slab subassemblies rather than spaces, or traversability scores that trade off furniture density against corridor width. These are the fields that determine whether a navigation prior is *robot-ready* or merely *geometry-complete*.

Why monolithic LLM calls are insufficient. A tempting fix is to hand the entire IFC file to a large model and ask for a robot summary of the building. Prior works show that LLMs can reason over BIM in principle – MCP4IFC [11], ifcMCP [12], and modular MCP-BIM architectures [13] expose IFC to LLMs for design authoring; Text2BIM [14] uses multi-agent LLMs for generative BIM; BIM2RDT [15] introduces agentic BIM for point-cloud alignment. But robot priors are a *safety-critical* downstream artifact: a hallucinated corridor, a fabricated door width, or a dropped storey can silently corrupt downstream reasoning and navigation decisions. None of the above methods provide mechanisms to guarantee that LLM enrichment never degrades the topological baseline it sits on.

Our proposal: staged agentic enrichment with a regression gate. We treat IFC-to-prior maps compilation as a two-layer problem. A deterministic backbone extracts structure (rooms, transitions, waypoints, occupancy). A separate *agentic* layer – a staged VLM pipeline with explicit per-stage responsibilities – annotates and enriches that structure with semantics the backbone cannot directly infer. Crucially, the agentic layer operates under two policies, `agent_structural_policy=none` and `agent_parameter_policy=fixed`, that forbid it from mutating topology or extraction parameters, and its output must pass a *topology regression gate* that compares candidate against baseline metrics; on regression, the pipeline falls back to the deterministic baseline. The resulting artifact is a versioned *navigation contract* bundling occupancy, waypoints, topology, semantics, confidence, and full provenance.

Fig. 1 frames the full vision: an offline agentic IFC-to-prior compiler (top) that feeds an online dual-agent navigation loop (bottom). This paper focuses on the *top half* – the agentic prior-generation pipeline and the contract it produces – which we claim is the load-bearing contribution. The online loop, while motivating, is deferred to future work.

Our contributions are the following.

- A three-layer robot-consumable navigation graph extracted top-down from IFC, with direct exports to Nav2, Open-RMF, GraphML, and GeoJSON.
- A tool-augmented LLM agent that tunes the extraction parameters for the backbone, writes semantic annotations into the graph, and produces a deterministic natural-language briefing for an on-board robot LLM.
- A study on four public building models, and showing that agent pipelines offer a significant boost in semantic coverage while preserving topological safety.

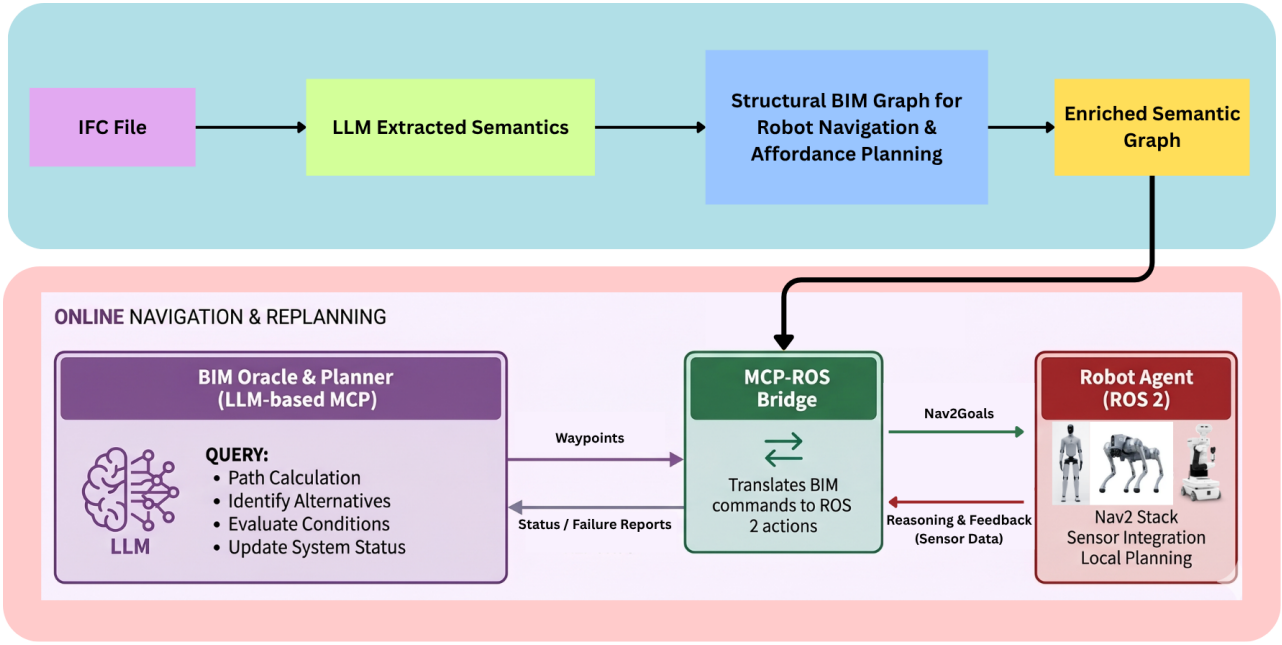


Fig. 1: RoboBIM system vision. **Top (this paper):** IFC is compiled into a multi-layer navigation contract by a staged LLM agent (Gemini 3 Flash), with a deterministic structural backbone and a policy-gated semantic-enrichment layer. **Bottom (outlook, Sec. ??):** the contract becomes the substrate for an online BIM-Oracle planner that communicates with a ROS 2 robot agent via an MCP bridge for failure-driven replanning. Contributions in this paper concern the offline agentic prior.

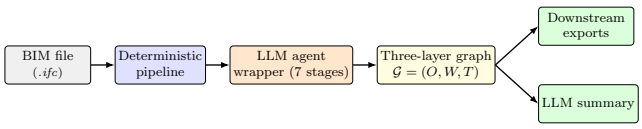


Fig. 2: System overview.

II. Related Work

IFC-to-robot priors (deterministic). Hamieh et al. [3], [16] extract IFC into weighted hypergraphs for cascade A*; Follini et al. [4] established the IFC-to-costmap pipeline; subsequent work automates IFC-to-waypoint extraction for construction robots [5], [10], [17]. The IFC-graph literature [6]–[9] formalizes topological graph generation from IFC. All these pipelines share a common ceiling: the semantics they expose are those already explicit in the IFC schema, and fields such as hazards, floor materials, and per-robot traversability are either null or populated only for standards-conforming building models. Our deterministic backbone sits in this line; our contribution is the agentic enrichment layer that compiles over it.

3D scene graphs and perception-based semantic priors. SayPlan [18], EmbodiedRAG [19], ContextMatters [20], and Taskography [21] build hierarchical 3D scene graphs from onboard sensing and use LLMs for high-level planning. Ray et al. [22] define structured interfaces for scene-graph reasoning. These systems solve the complementary problem: rich semantics *without* an a-priori map. We solve the dual: rich semantics *from* an a-priori map, with no perception required at enrichment time. Pix2G [23] bridges the two by pairing IFC with real-time scene-graph construction; our work targets the upstream stage of making the IFC itself semantically rich enough to seed such a bridge.

LLM-BIM integration (design-centric). MCP4IFC [11], ifcMCP [12], modular MCP-BIM architectures [13], and Text2BIM [14] validate that LLMs can operate over IFC via tool-calling interfaces, but target architectural design or model generation. BIM2RDT [15] introduces agentic BIM reasoning but applies it to point-cloud-to-BIM alignment. Kim et al. [24] use LLMs for construction robot task planning downstream of BIM. None of these provide a policy-gated, safety-constrained pipeline for compiling IFC into a robot navigation prior; this is our niche.

LLMs for affordance and capability-aware planning. SayCan [25] grounds LLM plans in robot affordances; AutoGPT+P [26] performs affordance-based task planning; VAMOS [27] and TIC-VLA [28] introduce capability-modulated navigation; AINav [29] and BrainBody-LLM [30] close the loop with adaptive replanning; IndoorR2X [31] coordinates indoor robots with LLM-driven planning. All assume semantic priors are *given*. Our navigation contract is designed as a drop-in input to such planners.

Positioning. Table I summarizes the gap. The distinguishing capabilities of our pipeline are (i) staged rather than monolithic LLM invocation, (ii) an explicit topology regression gate, and (iii) a multi-consumer contract output rather than a single bespoke format.

III. Methodology

A. Overview

The system has two main parts, a deterministic geometric backbone and a tool-augmented LLM agent wrapper. A

TABLE I: Positioning. RoboBIM targets agentic IFC-to-prior *compilation* with safety gates, orthogonal to both deterministic IFC pipelines and perception-based semantic priors.

Capability	**Deterministic IFC pipelines**	**3DSG + LLM planning**	**RoboBIM**
IFC as prior source	✓	✗	✓
Rich semantic annotation	✗	✓	✓
No perception required	✓	✗	✓
Staged LLM with bounded roles	✗	partial	✓
Topology regression gate	✗	✗	✓
Multi-consumer contract	partial	✗	✓
Per-robot traversability	✗	partial	✓

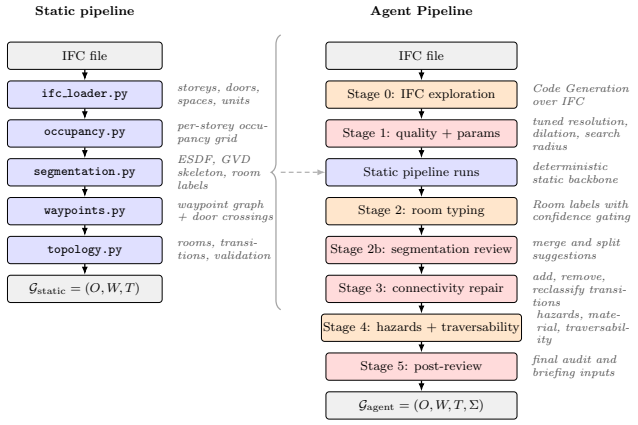


Fig. 3: Our static pipeline (left) vs our RoboBIM agent pipeline (right). The static pipeline is five deterministic Python modules that produce the geometric graph $\mathcal{G}_{\text{static}}$. The agent pipeline wraps the same backbone for semantic enrichment.

deterministic backbone first builds the geometric graph from the input building file, and a tool-augmented LLM agent then adds semantic annotations on top of that graph. Fig. 3 shows the two parts side by side, with the static backbone on the left and the agent pipeline on the right. The backbone takes a single IFC file as input and produces a three-layer graph of per-storey occupancy grids, a waypoint graph, and a room-and-transition topology. The agent then wraps the same backbone in seven stages. It probes the file through a sandboxed code-execution tool, tunes the backbone's extraction parameters, and writes semantic fields such as room types, hazards, traversability, and navigation policies back into the graph. The final output is the enriched graph together with Nav2, Open-RMF, GraphML, and GeoJSON exports, and a deterministic natural-language summary intended for the system prompt of an on-board robot LLM. The following subsections describe each part in turn.

B. Static pipeline

The static pipeline has five modules. First, the loader parses the IFC. It collects storeys, doors, and any room entities. The occupancy module then rasterises walls and columns per storey into a 2D grid at specified resolution. From this grid, the segmentation module computes the Euclidean signed distance field and extracts the medial-axis (GVD) skeleton. It then splits the free space into rooms using door-aware connected components. When no room entities like spaces are present, it falls back to a purely geometric pass driven by walls and door positions alone. The waypoints module traces the skeleton into a sparse graph and injects an additional waypoint at every door crossing, so that the graph follows the building's access layout. Finally, the topology module constructs the room-and-transition graph and attaches materials from property sets where present. It also runs a cross-storey consistency check that flags stair or elevator edges whose endpoints do not align between floors. The result is the geometric graph $\mathcal{G}_{\text{static}}$, produced without any LLM intervention.

C. Agent Pipeline

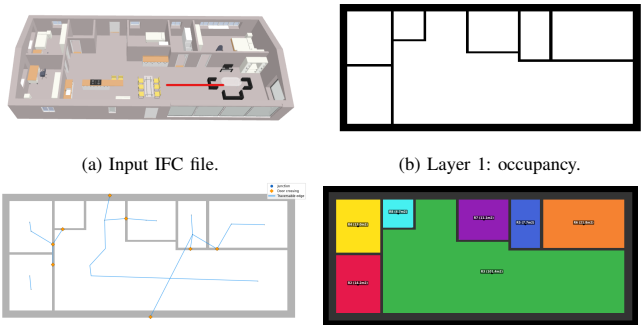
The agent pipeline wraps the same backbone with LLM stages. Each stage reads a shared Pipeline Context and writes its findings back into it. In Stage 0 the agent receives a compact IFC summary and a code-execution tool. It generates Python snippets against the loaded IFC and is instructed to use the IfcOpenShell toolkit. The snippets query element counts, relationship entities, and property sets, and what to query is fully controlled by the LLM. Their findings are used in every downstream stage. In Stage 1 the agent reads the same summary together with the Stage 0 findings. It outputs extraction parameters such as grid resolution, door-closing dilation radius, minimum room area, and door search radius. The agent cannot edit the backbone code itself, only the parameters that the code runs with. The backbone is deterministic, so any change in the output geometry must come from these parameters. The agent therefore tunes for better extraction quality without touching geometry or topology directly. The backbone then runs with the tuned parameters and produces the geometric graph. Stage 2 assigns room types. A regex and property-set prefilter resolves the easy cases first. The remaining rooms on a given storey are sent to the LLM in one batched prompt, so that cross-room context is visible in a single query. This is where the LLM is most useful. Much of the semantic information in a building file is not directly attached to rooms, but can be inferred from the context of the storey. For example, an unlabelled room with many door transitions and bordered by rooms marked as offices can usually be identified as a corridor. Writing deterministic rules to cover every such case is challenging, whereas the LLM can infer the missing labels from context and apply them across the storey. The resulting labels are

useful for a planner. A robot may want to treat corridors differently from offices, or know that a kitchen is likely to have hot surfaces and fragile objects.

Stage 2b then reviews the segmentation quality. The agent can suggest room merges or splits based on the geometry and the door placements. Stage 3 reviews the topology. The agent can suggest edits to the room-transition graph, such as adding a missing door transition, removing one, or reclassifying a transition type between door, opening, and landing. Stages 2b and 3 are therefore the only stages where the agent can change connectivity structure. Stage 4 annotates hazards, materials, and traversability scores for each room. The agent can read any part of the IFC and any part of the extracted graph when making these annotations. Finally, Stage 5 produces a natural-language audit. It reads the final graph together with the full agent trace of all previous stages. The output is a compact summary that lists key information about each room (function, occupancy, hazards, floor material, traversability) and also derives high-level policies such as which doors are likely to be bottlenecks and which storeys are non-navigable. This summary is designed to be pasted into the system prompt of an on-board robot LLM. The robot's reasoning is then grounded in the same prior that the agent built, without any additional LLM call at runtime.

D. Output Graphs

The output graph has three linked layers, shown in Fig. 4 with the input IFC file for reference. The static pipeline produces the geometric part of this graph. Layer 1 is a per-storey occupancy grid at 5 cm resolution, in which each cell is free, occupied, or unknown. Layer 2 is a per-storey waypoint graph derived from the Generalised Voronoi Diagram of the free space, and each edge carries a clearance value that decides whether a robot of a given radius can traverse it. Layer 3 is the topological graph of rooms and transitions, where transitions are doors, wall openings, and stair or elevator landings. Cross-layer links attach every room to its waypoint cluster and every door to its crossing waypoint, so that a planner can descend from a room-level plan to a metric path. Every room node also reserves space for semantic fields, namely a room-type label with confidence and source, a list of hazard flags, a floor-material string, and a traversability score in $[0, 1]$, but the static baseline leaves all of these fields empty. The agent pipeline preserves every layer and populates these reserved fields. It assigns room types from the Stage 2 prefilter and the batched LLM call, hazard flags and traversability scores from Stage 4, and floor materials where the file or the surrounding context allows them to be inferred. It also derives building-level navigation policies, such as the tightest door-width invariant and the storeys that contain no connected transitions, which attach to the graph rather than to any single room. The static and agent outputs therefore share the same geometric graph, and differ only in whether the semantic and policy fields are filled in.



(a) Input IFC file. (b) Layer 1: occupancy.

(c) Layer 2: waypoints. (d) Layer 3: rooms.

Fig. 4: Input IFC file and the three linked layers on BasicHouse, ground floor. Top-left: input IFC rendering. Top-right: Layer 1, 5 cm occupancy grid. Bottom-left: Layer 2, GVD-derived waypoint graph with clearance-annotated edges. Bottom-right: Layer 3, room-and-transition topology. Cross-layer links (not drawn) connect each room to its waypoint cluster and each door to its crossing waypoint.

IV. Experiments

A. Dataset and Experimental Setup

We evaluate on four public building models: 4351, BasicHouse, digital_hub, and samuel_macalister_sample_house. These cover residential and office layouts with 14 to 64 rooms. BasicHouse has no existing room entities and therefore exercises the geometric fallback, in which the agent infers room semantics from geometry alone. Appendix G provides the file history, licenses, and statistics for each model. We run both pipelines on every building in our dataset. The static pipeline serves as the baseline, and the agent pipeline uses Gemini 3.0 Flash preview for every agent run. Every reported number is a single-run measurement. We compare the two pipelines along three dimensions: the geometric graph they produce, the semantic fields they populate, and the computational cost of the agent pass.

B. Results

Table II summarises the per-IFC results. We report enrichment on five axes that correspond to the Semantic enrichment columns of Table II. Type counts the rooms with an inferred room-type label such as kitchen, corridor, or office and not just there is a room. The label comes from the regex and property-set prefilter in Stage 2, or from the batched per-storey LLM call when the prefilter abstains. With hazard counts rooms that carry at least one hazard flag from four categories observed in the data: wet area, obstacle-dense, high traffic, and narrow passage. Traversability counts rooms with a numeric score in $[0, 1]$ that captures how freely a standard mobile base can move through the space. Material rooms with an inferred floor-material string, such as concrete or tile. Policy counts building-level navigation policies derived from the enriched graph, for example the tightest door-width invariant or storeys flagged as non-navigable. The agent also assigns traversability scores for every navigable room, floor-material strings, and building-level navigation policies. On BasicHouse, the static baseline labels no rooms because the file carries no room entities, while the agent labels all seven navigable rooms from geometry and adjacency alone. On

TABLE II: Per-building results on four building models. The Geometry columns are identical for the static and agent pipelines and are therefore listed once. The Semantic enrichment columns are written as static → agent, so both the baseline value and the value after the agent pass appear in the same cell.

	Geometry (preserved)				Semantic enrichment (static → agent)					Agent cost	
Model	storeys	rooms	conn. trans.	waypoints	type	w/ hazard	traversability	material	policy	time (s)	tok. in/out
4351	5	22	10	90	1 → 1	0 → 3	0 → 22	0 → 5	0 → 5	294	285k/32k
BasicHouse	2	14	6	32	0 → 7	0 → 7	0 → 14	0 → 7	0 → 6	489	200k/24k
digital_hub	3	64	60	304	38 → 64	0 → 0	0 → 64	0 → 0	0 → 1	423	343k/32k
macalister	6	31	9	392	12 → 17	0 → 9	0 → 31	0 → 17	0 → 6	407	348k/41k

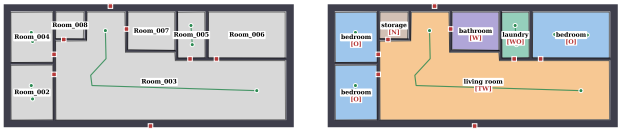


Fig. 5: Floor 0 in BasicHouse file. Left: static output and Right: agent output. Rooms coloured by inferred type with hazard codes in brackets.

digital_hub, the agent reaches 64 of 64 rooms, closing the 38-of-64 gap left by the regex baseline.

The geometric graph is identical for the two pipelines, so the storey, room, connected-transition, and waypoint counts appear only once in Table II. No agent edit reduced connectivity below the baseline on any of the four runs, so the agent pass remained topologically safe in these experiments. Agent wall-clock time ranges from 294s to 489s per IFC, with a mean of 403s. Input tokens per building fall between 200k and 348k, and output tokens between 24k and 41k. Stage 0 and Stage 2 dominate the time budget, and per-storey batching keeps the total LLM call count between 7 and 15 per run. Appendix G and Fig. 6 break down the cost per stage and per IFC.

C. Qualitative Analysis of the Agent Pipeline

Fig. 5 shows the static and agent outputs on Floor 0 of BasicHouse. The static pipeline shows uniformly grey rooms, a waypoint graph, and geometric room indices. The agent pipeline colours rooms by inferred type and marks per-room hazards. Hazard codes are W for wet, O for obstacle-dense, T for high traffic, and N for narrow passage. A short excerpt of the generated briefing, covering four rooms and the derived navigation policies, is reproduced in Listing 1 in the appendix. It lists the inferred room type, hazards, floor material, traversability, and policy limits in one place, and the static baseline populates none of these fields.

D. On-board Querying with Agentic Priors

The exports and the briefing can be served to a robot-side LLM through the Model Context Protocol [32]. If the local planner reports a blocked door or an occluded corridor, an on-board agent can query alternatives that satisfy width, hazard, and traversability constraints, and the robot can reroute using this prior instead of re-exploring the full global state space. This building context can be delivered to the on-board LLM through its context window or through retrieval-augmented generation (RAG). The static pipeline provides geometry alone, and does not populate the semantic and policy fields that such queries require.

V. Discussion and Extensions

The prior is only as accurate as the IFC file. It is the as-designed state of the building, not the as-built one. Closing that gap is the main direction that this work opens. Three extensions are close at hand.

Fusion with perception is a direct extension. Our graph is the ground-truth target that Hydra-class systems try to reconstruct. A fusion layer can use the BIM prior as a stable reference and update it with DAAAM [**?**]-style captions or online SLAM. The result would be an as-built prior that preserves an authoritative starting point.

Adding graph layers is another direct extension. The output schema already reserves fields for two more layers. A containment layer can list the objects that each room contains, and a boundary layer can encode the materials of each room's surfaces. Both layers are available in the IFC schema but are not yet extracted by the pipeline. One extraction pass per layer would support object-aware and sensor-aware planning. For example, a planner could penalise glass curtain walls for LiDAR-based robots.

Adversarial and large-scale evaluation is still needed. The topology regression gate has not been tested under hostile prompting. In our buildings, confidence stayed below threshold on every risky edit. A targeted bad-merge prompt would stress-test the gate. A larger building files would test the per-storey scaling claim.

VI. Conclusion

We presented RoboBIM, an IFC-to-navigation-graph pipeline that treats IFC processing as semantic enrichment over a deterministic geometric backbone. The backbone produces a three-layer graph of occupancy grids, waypoints, and a room-transition topology, and a tool-augmented LLM agent then writes room types, hazard flags, traversability scores, floor materials, and building-level navigation policies back into the graph. On the four-building files, the agent raises room type coverage, adds hazard annotations where the baseline has none, scores traversability for every navigable room, and preserves connected topology across all runs. The natural next step is to close the gap between the as-designed model and the as-built state through perception fusion, richer graph layers from the IFC relationship entities, and an on-board query agent that reuses the same tool interface, so that the robot can query the prior as its plan meets the real building.

REFERENCES

[1] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, "Kimera: From SLAM to spatial perception with 3D dynamic scene graphs," in *International Journal of Robotics Research*, 2021.

[2] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3D scene graph construction and optimization," in *RSS*, 2024.

[3] A. Hamieh, A. A.-R. Moughlbay, B. Marche, and C. Francois, "Semantic navigation using building information on construction sites," *arXiv preprint arXiv:2104.10296*, 2021.

[4] C. Follini, M. Terzer, C. Marcher, A. Giusti, and D. T. Matt, "BIM-integrated collaborative robotics for application in building construction," in *Proceedings of the International Symposium on Automation and Robotics in Construction*, 2020.

[5] K. Kim and M. Peavy, "BIM-based semantic building world modeling for robot task planning," *Automation in Construction*, vol. 138, 2022.

[6] T.-A. Teo and K.-H. Cho, "i-GIT: Intelligent generation of indoor topology from IFC models," *ScienceDirect*, 2018.

[7] Y.-H. Lin *et al.*, "Generating straight skeleton-based navigation networks with IFC," *ScienceDirect*, 2020.

[8] J. Zhu *et al.*, "A new graph-based approach for building information querying," in *CIB W78 Conference*, 2024.

[9] "Semantics-based connectivity graph for indoor pathfinding powered by IFC-Graph," *ScienceDirect*, 2025.

[10] Construction Robotics Authors, "Improving autonomous robotic navigation using IFC files," *Construction Robotics*, 2023.

[11] B. K. Nithyanantham *et al.*, "MCP4IFC: IFC-based building design using LLMs," *arXiv preprint arXiv:2511.05533*, 2025.

[12] J.-R. Lin and P. Pan, "ifcMCP: Enabling LLM agents to talk with IFC files," 2025.

[13] "A modular reference architecture for MCP-servers for agentic BIM interaction," *arXiv preprint arXiv:2601.00809*, 2025.

[14] C. Du *et al.*, "Text2BIM: Generating building models using a LLM-based multi-agent framework," *arXiv preprint arXiv:2408.08054*, 2024.

[15] "BIM2RDT: Building information models to robot-ready site digital twins," *arXiv preprint arXiv:2509.20705*, 2025.

[16] A. Hamieh, A. A.-R. Moughlbay, and B. Marche, "Intuitive BIM-aided robotic navigation and assets localization," *Frontiers in Robotics and AI*, 2025.

[17] S. Kim, H. A. Pham, J. Jang, and G. Lee, "Development of BIM-integrated construction robot task planning system," *Automation in Construction*, 2021.

[18] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, "SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning," in *CoRL*, PMLR 229, 2023.

[19] M. Booker, C. Agia, P. Sermanet, and A. Z. Ren, "EmbodiedRAG: Dynamic 3D scene graph retrieval for efficient robot task planning," *arXiv preprint arXiv:2410.23968*, 2024.

[20] "ContextMatters: Relaxing goals with LLMs for feasible 3D scene planning," *arXiv*, 2026.

[21] C. Agia *et al.*, "Taskography: Evaluating robot task planning over large 3D scene graphs," in *CoRL*, PMLR, 2022.

[22] A. Ray *et al.*, "Structured interfaces for automated reasoning with 3D scene graphs," *arXiv preprint arXiv:2510.16643*, 2025.

[23] A. Longo *et al.*, "Pixels-to-graph: Real-time integration of BIM and scene graphs," *arXiv preprint arXiv:2506.22593*, 2025.

[24] K. Kim *et al.*, "Framework for LLM-enabled construction robot task planning," *Robotics*, vol. 14, no. 9, p. 117, 2025.

[25] M. Ahn, A. Brohan, N. Brown, *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.

[26] T. Birr *et al.*, "AutoGPT+P: Affordance-based task planning using LLMs," in *RSS*, 2024.

[27] "VAMOS: A hierarchical VLA model for capability-modulated navigation," *arXiv preprint arXiv:2510.20818*, 2025.

[28] Z. Huang *et al.*, "TIC-VLA: A think-in-control VLA model for robot navigation," *arXiv preprint arXiv:2602.02459*, 2026.

[29] "AINav: Interactive navigation via LLM-based skill tree and adaptive replanning," *arXiv preprint arXiv:2503.22942*, 2025.

[30] "BrainBody-LLM: Grounding LLMs for robot task planning with closed-loop feedback," *Advanced Robotics Research*, 2025.

[31] "IndoorR2X: Indoor robot-to-everything coordination with LLM-driven planning," *arXiv preprint arXiv:2603.20182*, 2026.

[32] Anthropic, "Model context protocol (MCP) specification." `https://modelcontextprotocol.io`, 2024.

APPENDIX

This appendix expands the seven agent stages introduced in Section III. For each stage we describe the inputs it reads, the output it produces, and the conditions under which its proposals are written back into the graph.

A. Stage 0: building exploration

The agent receives a compact summary of the input building file that lists entity counts, storeys, doors, and representative property sets. It is also given a sandboxed Python runner with read-only access to the loaded building model through IfcOpenShell [**?**]. The runner enforces a hard timeout and blocks imports of non-standard libraries and file writes. The agent responds with free-form findings, which are parsed into evidence items and stored in the Pipeline Context.

B. Stage 1: quality analysis and parameter tuning

The agent reads the building summary together with the Stage 0 findings. It returns a structured object with a completeness score, a recommended strategy, and five numeric parameters: grid resolution, dilation radius, minimum room area, door search radius, and an include-openings flag. All numeric parameters are clamped to safe ranges. A parameter policy controls the write-back behaviour. In the apply mode, the parameters are used directly. In the suggest mode, they are used only if they lie inside the clamp range. In the fixed mode, which is the default, they are logged but not used.

C. Stage 2: room-type inference

A regex prefilter types rooms with obvious names, such as corridor, CX, or WC. An element-set prefilter handles rooms with matching content, for example a room with two or more sanitary terminals is treated as a bathroom. Rooms resolved by either prefilter skip the LLM. The remaining rooms on a given storey are sent to the LLM in a single batched prompt. A newly assigned type requires a confidence of at least 0.80, and overriding an existing type requires a confidence of at least 0.75.

D. Stage 2b: segmentation review

The agent reads the room list and the transitions, then proposes merge or split directives. Merges with a confidence of at least 0.70 are applied at the topology layer: the bounding boxes are combined, the transitions are reassigned, and the absorbed room is removed. Split directives are recorded only, because they require label-map access that is not preserved after extraction.

E. Stage 3: connectivity validation and repair

The agent produces a list of issues, such as isolated rooms or suspicious connections. It then outputs a repair plan with three directive kinds: add_connection, remove_connection, and reclassify. Directives with a confidence of at least 0.70 are applied. The topology regression gate then checks the

resulting connectivity count, and any edit that reduces it is rolled back.

F. Stage 4: hazards and traversability

For each room, the agent writes a list of hazard flags from a controlled vocabulary, a floor-material string, a traversability score in $[0, 1]$, a short robot-obstacles string, and a prose note. The stage reuses the prefilter and the per-storey batching introduced in Stage 2.

G. Stage 5: post-extraction review

The agent reads the final enriched graph and returns a short audit that contains an overall quality score, a completeness score, a gaps list, and improvement suggestions. The audit is stored in the Pipeline Context and is not applied to the graph.

The four building models used in the main evaluation are all public and redistributable, and each is taken directly from its original repository. 4351[1] and samuel_macalister_sample_house[2] come from the BIMserver TestFiles collection. digital_hub[3] comes from the RWTH E3D DigitalHub release. BasicHouse[4] comes from the BIM-Whale samples repository, which is a small collection of MIT-licensed test buildings. Together, these sources span several design disciplines and both major revisions of the IFC file format.

Table III lists the upstream origin, the license, and the intended building use for each model. Table IV lists raw statistics extracted from each file: the schema version, the on-disk file size, and the total counts of storeys, doors, rooms, and walls. These counts are independent of our pipeline and reflect the authoring state of each file.

TABLE III: Provenance of each building model in the corpus.

Model	Upstream source	License	Use
4351	BIMserver TestFiles	GPL v3	office
BasicHouse	BIM-Whale Samples	MIT	residential
digital_hub	RWTH E3D DigitalHub	MIT	mixed-use
macalister	BIMserver TestFiles	GPL v3	residential

TABLE IV: Raw building-file statistics extracted from each model. Counts are the total number of each element type present in the file, before any geometric extraction. The rooms column shows how many rooms are explicitly annotated by the author; a zero means the pipeline must recover rooms geometrically.

Model	Schema	Size (MB)	Storeys	Doors	Rooms	Walls
4351	IFC2X3	7.3	5	17	6	43
BasicHouse	IFC2X3	51.7	2	8	0	13
digital_hub	IFC4	9.0	3	64	64	178
macalister	IFC4	25.4	6	16	14	48

The four models cover a range of size and authoring quality. digital_hub is the densest building in the corpus, with 64 annotated rooms and 64 doors on three storeys, and tests the pipeline on a richly annotated model. 4351 is a 5-storey office with only 6 rooms annotated, so the agent must infer room types on a partially annotated file. BasicHouse has no room annotations at all, so segmentation falls through to the geometric GVD pass described in Section III. It is also the largest file on disk, because it carries detailed geometry for a small floor plan. samuel_macalister_sample_house is a 6-storey residential model with mechanical and plumbing content, and it stresses the cross-storey consistency check because the storey count is the largest in the corpus.

Fig. 6 stacks the seven stage timings per building and overlays the input-token count on a secondary axis. Table **??** lists the underlying per-stage wall-clock times. Stage 0 is always the first LLM stage. Its time scales with the complexity of the input file, because the agent controls how many code-execution snippets to issue. Stage 2, room-type inference, scales with the storey count, because it is batched per storey. Stage 2b, segmentation review, is the largest single cost on BasicHouse, where the lack of annotated rooms forces a richer geometric review. Stage 3 is the largest cost on digital_hub, which has the densest connectivity graph in the corpus.

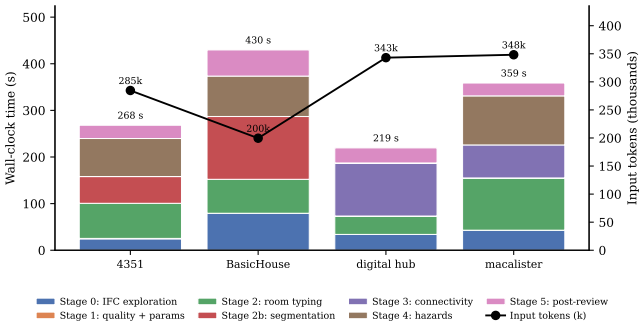


Fig. 6: Per-IFC agent wall-clock time stacked by stage (bars, left axis), with the input-token count overlaid as a black line (right axis). Stage 0, Stage 2, Stage 2b, and Stage 4 dominate the time budget. Confidence-gated per-storey batching keeps the total LLM call count between 7 and 15 per run.

Table V reports the LLM usage per building. Both the call count and the token usage are dominated by Stage 2 and Stage 4 on the smaller buildings. On BasicHouse, Stage 0 dominates, because the agent chose to issue more exploration snippets than on the other three buildings.

TABLE V: LLM usage per building.

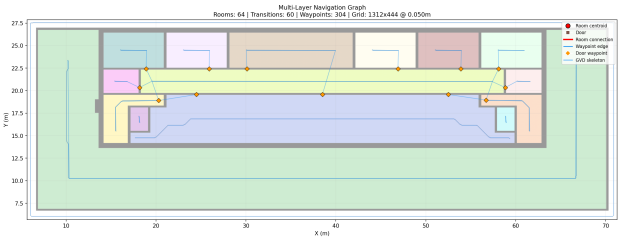
Model	LLM calls	Code execs	Tokens in (k)	Tokens out (k)
4351	13	28	285	32
BasicHouse	9	15	200	24
digital_hub	7	16	343	32
macalister	15	27	348	41
total	44	86	1176	129

This appendix collects additional per-storey visualisations produced by the agent pipeline on two of the four buildings. Each panel is a composite view written by the pipeline that

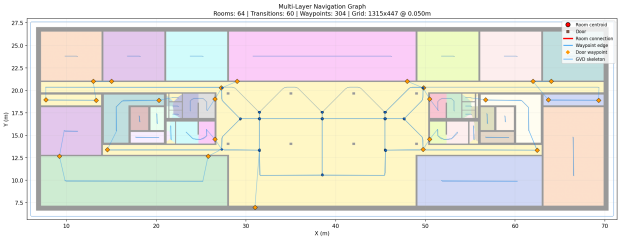
[1] https://github.com/opensourceBIM/TestFiles
[2] https://github.com/opensourceBIM/TestFiles
[3] https://github.com/RWTH-E3D/DigitalHub
[4] https://github.com/andrewisen/bim-whale-ifc-samples/tree/master/BasicHouse

overlays the occupancy grid, the waypoint graph, and the room segmentation on a single storey.

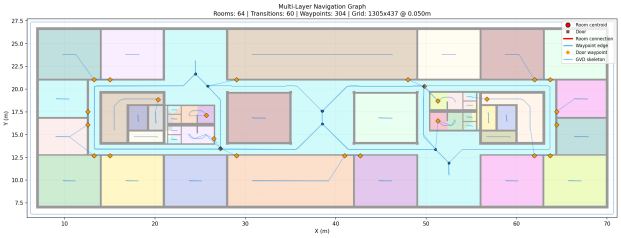
Fig. 7 shows the three storeys of digital_hub. The building has annotated rooms on every floor, so the segmentation follows the authored partitions and the waypoint graph densifies cleanly around the shared corridor on each level.



(a) Basement (B01_OKRD).



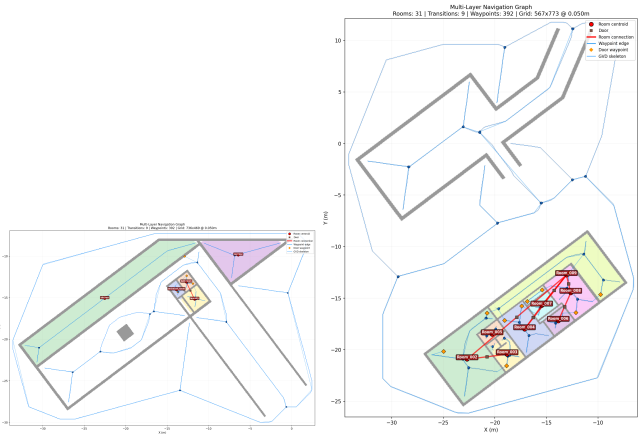
(b) Ground floor (E00_OKRD).



(c) First floor (E01_OKRD).

Fig. 7: Per-storey composite for digital_hub. Each panel overlays the occupancy grid (grey), the waypoint graph (blue nodes and edges), and the room segmentation (coloured regions) on one storey of the building.

Fig. 8 shows two of the six storeys in samuel_macalister_sample_house. The dwelling has a residential layout that varies across floors, and the waypoint graph contracts around the stair and landing regions on each level.



(a) Level 1. (b) Level 2.

Fig. 8: Composite view on two storeys of samuel_macalister_sample_house. The pipeline produces a consistent waypoint graph across levels, with room boundaries drawn from the authored room partitions where available.

Listing 1: Short summary of the BasicHouse briefing derived from the agent trace. The full agent trace is larger and includes stage-by-stage details.

```
Room_003: living_room, concrete, trav=0.80
  hazards: [high_traffic, wet_area]
  notes:   circulation hub, 8 door transitions

Room_005: laundry, concrete, trav=0.50
  hazards: [wet_area, obstacle_dense]
  notes:   narrow channels around appliances

Room_007: bathroom, concrete, trav=0.60
  hazards: [wet_area]
  notes:   constrained layout, plumbing fixtures

Room_008: storage, concrete, trav=0.90
  hazards: [narrow_passage]
  notes:   small walk-in closet

derived_policies:
  tightest_door_m:       0.91
  width_limit_m:         0.81
  caution_rooms:         [Room_003, Room_005, Room_007]
  non_navigable_storeys: [Floor_1]
```